

Stack Applications: NGE, PSE, and Expression Conversion

These are important **stack-based problems** in Data Structures. The key idea is that a stack can avoid repeatedly scanning the array or expression, often reducing the time complexity from $O(n^2)$ to $O(n)$.

1. Next Greater Element (NGE)

Definition

For each element in an array, the **Next Greater Element** is the first element to its **right** that is greater than it.

If no greater element exists, the answer is usually `-1`.

Example

Array:

[4, 5, 2, 10, 8]

Next Greater Element:

[5, 10, 10, -1, -1]

Explanation:

Element	Elements to its right	NGE
4	5, 2, 10, 8	5
5	2, 10, 8	10
2	10, 8	10
10	8	-1
8	—	-1

1.1 Brute Force Approach

For every element, scan all elements to its right.

Algorithm

```

for i = 0 to n-1
    answer[i] = -1

    for j = i+1 to n-1
        if arr[j] > arr[i]
            answer[i] = arr[j]
            break

```

Complexity

- Time: $O(n^2)$
- Space: $O(n)$ for the output

1.2 NGE Using Stack

A stack allows us to solve NGE in $O(n)$.

Important idea

Traverse the array from **right to left**.

For every element:

1. Remove all stack elements that are `<= current`.
2. The remaining top element is the NGE.
3. Push the current element.

Algorithm

```

NGE(arr)

1. Create empty stack
2. Create result array

3. Traverse from right to left:

    while stack is not empty AND stack.top <= arr[i]
        pop stack

    if stack is empty
        result[i] = -1
    else
        result[i] = stack.top

    push arr[i]

```

1.3 Example

```
arr = [4, 5, 2, 10, 8]
```

Process from right to left:

Current	Stack before	Pop	NGE	Stack after
8	[]	—	-1	[8]
10	[8]	8	-1	[10]
2	[10]	—	10	[10, 2]
5	[10, 2]	2	10	[10, 5]
4	[10, 5]	—	5	[10, 5, 4]

Result:

```
[5, 10, 10, -1, -1]
```

1.4 Python Code – NGE

Python

Run

```
def next_greater_element(arr):
    n = len(arr)
    result = [-1] * n
    stack = []

    for i in range(n - 1, -1, -1):

        # Remove elements that cannot be the answer
        while stack and stack[-1] <= arr[i]:
            stack.pop()

        # Top of stack is the next greater element
        if stack:
            result[i] = stack[-1]

        # Push current element
        stack.append(arr[i])
```

```
        return result

arr = [4, 5, 2, 10, 8]

print("Array:", arr)
print("NGE :", next_greater_element(arr))
```

Output

```
Array: [4, 5, 2, 10, 8]
NGE  : [5, 10, 10, -1, -1]
```

Complexity Analysis

Although there is a `while` loop inside the `for` loop, the complexity is **$O(n)$** , not $O(n^2)$.

Why?

Each element:

- is pushed onto the stack once
- can be popped at most once

Therefore:

```
Total pushes = O(n)
Total pops   = O(n)

Time = O(n)
Space = O(n)
```

2. Previous Smaller Element (PSE)

Definition

For every element, the **Previous Smaller Element** is the first element to its **left** that is smaller than it.

If no smaller element exists, return `-1`.

Example

```
Array:
```

```
[4, 5, 2, 10, 8]
```

Previous Smaller:

```
[-1, 4, -1, 2, 2]
```

Explanation:

Element	Elements to left	PSE
4	—	-1
5	4	4
2	4, 5	-1
10	4, 5, 2	2
8	4, 5, 2, 10	2

2.1 Brute Force Approach

```
for i = 0 to n-1

    answer[i] = -1

    for j = i-1 down to 0

        if arr[j] < arr[i]
            answer[i] = arr[j]
            break
```

Complexity

```
Time   = O(n2)
Space  = O(n)
```

2.2 PSE Using Stack

Traverse from **left to right**.

For every element:

1. Remove all elements from the stack that are `>= current`.

2. The remaining top is the previous smaller element.
3. Push current element.

Algorithm

PSE(arr)

1. Create empty stack
2. Create result array

3. Traverse from left to right:

```
while stack is not empty AND stack.top >= arr[i]
    pop stack
```

```
if stack is empty
    result[i] = -1
```

```
else
    result[i] = stack.top
```

```
push arr[i]
```

4. Return result

2.3 Example

arr = [4, 5, 2, 10, 8]

Current	Stack before	Pop	PSE	Stack after
4	[]	—	-1	[4]
5	[4]	—	4	[4, 5]
2	[4, 5]	5, 4	-1	[2]
10	[2]	—	2	[2, 10]
8	[2, 10]	10	2	[2, 8]

Result:

[-1, 4, -1, 2, 2]

2.4 Python Code – PSE

Python

Run

```
def previous_smaller_element(arr):
    n = len(arr)
    result = [-1] * n
    stack = []

    for i in range(n):

        # Remove elements that are not smaller
        while stack and stack[-1] >= arr[i]:
            stack.pop()

        # Top is previous smaller element
        if stack:
            result[i] = stack[-1]

        # Push current element
        stack.append(arr[i])

    return result

arr = [4, 5, 2, 10, 8]

print("Array:", arr)
print("PSE :", previous_smaller_element(arr))
```

Output

```
Array: [4, 5, 2, 10, 8]
PSE  : [-1, 4, -1, 2, 2]
```

3. NGE vs PSE

Feature	Next Greater Element	Previous Smaller Element
Direction	Right	Left
Traversal	Right → Left	Left → Right
Stack condition	Pop <code><= current</code>	Pop <code>>= current</code>
Stack maintains	Decreasing candidates	Increasing candidates

Feature	Next Greater Element	Previous Smaller Element
Time	$O(n)$	$O(n)$
Space	$O(n)$	$O(n)$

Easy way to remember

NGE $\rightarrow$ look RIGHT $\rightarrow$ traverse RIGHT to LEFT

PSE $\rightarrow$ look LEFT $\rightarrow$ traverse LEFT to RIGHT

4. Expression Conversion

There are three major expression notations:

Infix

A + B

Operator is between operands.

Prefix

+ A B

Operator comes before operands.

Postfix

A B +

Operator comes after operands.

5. Infix $\rightarrow$ Postfix

Example

A + B * C

Since `*` has higher precedence:

`A + (B * C)`

Postfix:

`ABC*+`

Algorithm

`INFIX_TO_POSTFIX(expression)`

1. Create empty stack.
2. Create empty output.
3. Scan expression from left to right.
4. If operand:
 Add to output.
5. If '(':
 Push onto stack.
6. If ')':
 Pop until '(' is found.
 Remove '('.
7. If operator:
 While stack is not empty
 and top has higher/equal precedence:
 Pop to output.

 Push current operator.
8. Pop all remaining operators.
9. Return output.

5.1 Operator Precedence

Operator	Precedence
----------	------------

<code>^</code>	3
----------------	---

Operator	Precedence
* / %	2
+ -	1

Parentheses have special handling.

5.2 Python Code – Infix to Postfix

Python

Run

```
def precedence(op):
    if op == '^':
        return 3
    elif op in ('*', '/', '%'):
        return 2
    elif op in ('+', '-'):
        return 1
    return 0

def infix_to_postfix(expression):
    stack = []
    result = []

    for ch in expression:

        # Ignore spaces
        if ch.isspace():
            continue

        # Operand
        if ch.isalnum():
            result.append(ch)

        # Opening parenthesis
        elif ch == '(':
            stack.append(ch)

        # Closing parenthesis
        elif ch == ')':
            while stack and stack[-1] != '(':
                result.append(stack.pop())

            if stack:
                stack.pop() # Remove '('
```

```

        # Operator
    else:
        while (stack
                and stack[-1] != '('
                and precedence(stack[-1]) >= precedence(ch)):
            result.append(stack.pop())

        stack.append(ch)

    # Empty the stack
    while stack:
        result.append(stack.pop())

    return ''.join(result)

expressions = [
    "A+B*C",
    "(A+B)*C",
    "A+B*(C-D)",
    "(A+B)*(C-D)"
]

for expression in expressions:
    print(expression, "->", infix_to_postfix(expression))

```

Output

```

A+B*C -> ABC*+
(A+B)*C -> AB+C*
A+B*(C-D) -> ABCD-*+
(A+B)*(C-D) -> AB+CD-*

```

6. Infix → Prefix

One common method is:

1. Reverse the infix expression.
2. Swap (and) .
3. Convert to postfix.
4. Reverse the result.

Example

(A+B)*C

Prefix:

*+ABC

Python Code

Python

Run

```
def infix_to_prefix(expression):
    # Reverse expression and swap parentheses
    reversed_expression = []

    for ch in expression[::-1]:
        if ch == '(':
            reversed_expression.append(')')
        elif ch == ')':
            reversed_expression.append('(')
        else:
            reversed_expression.append(ch)

    reversed_expression = ''.join(reversed_expression)

    # Convert reversed expression to postfix
    postfix = infix_to_postfix(reversed_expression)

    # Reverse postfix to get prefix
    return postfix[::-1]

expressions = [
    "A+B*C",
    "(A+B)*C",
    "A+B*(C-D)",
    "(A+B)*(C-D)"
]

for expression in expressions:
    print(expression, "->", infix_to_prefix(expression))
```

Expected results:

```
A+B*C -> +A*BC
(A+B)*C -> *+ABC
A+B*(C-D) -> +A*B-CD
(A+B)*(C-D) -> *+AB-CD
```

Note: For a production-quality converter, associativity of `^` should also be handled carefully. The simple version above is sufficient for basic DS examples using `+`, `-`,

7. Postfix → Infix

Postfix:

ABC*+

Scan from left to right.

Rules

Operand:

PUSH

Operator:

operand2 = POP

operand1 = POP

expression = (operand1 operator operand2)

PUSH expression

Example

ABC*+

Symbol	Stack
A	A
B	A, B
C	A, B, C
*	A, (B*C)
+	(A+ (B*C))

Answer:

(A+ (B*C))

Python Code

Python

Run

```
def postfix_to_infix(expression):
    stack = []

    for ch in expression:

        if ch.isalnum():
            stack.append(ch)

        else:
            operand2 = stack.pop()
            operand1 = stack.pop()

            new_expression = (
                "(" + operand1 + ch + operand2 + ")"
            )

            stack.append(new_expression)

    return stack[-1]

postfix = "ABC*+"

print("Postfix:", postfix)
print("Infix:", postfix_to_infix(postfix))
```

Output:

Postfix: ABC*+
Infix: (A+ (B*C))

8. Prefix → Infix

Prefix:

*+ABC

Scan from **right to left**.

Rules

Operand:

PUSH

Operator:

```
operand1 = POP
operand2 = POP

expression = (operand1 operator operand2)

PUSH expression
```

Example

*+ABC

Result:

((A+B)*C)

Python Code

Python

Run

```
def prefix_to_infix(expression):
    stack = []

    for ch in expression[::-1]:

        if ch.isalnum():
            stack.append(ch)

        else:
            operand1 = stack.pop()
            operand2 = stack.pop()

            new_expression = (
                "(" + operand1 + ch + operand2 + ")"
            )

            stack.append(new_expression)
```

```
        return stack[-1]

    prefix = "*+ABC"

    print("Prefix:", prefix)
    print("Infix:", prefix to infix(prefix))
```

Output:

```
Prefix: *+ABC
Infix: ((A+B)*C)
```

9. Postfix → Prefix

Postfix:

```
ABC*+
```

Scan from left to right.

For an operator:

```
operand2 = POP
operand1 = POP

result = operator + operand1 + operand2

PUSH result
```

Python Code

Python

Run

```
def postfix_to_prefix(expression):
    stack = []

    for ch in expression:

        if ch.isalnum():
            stack.append(ch)

        else:
            operand2 = stack.pop()
            operand1 = stack.pop()
```

```

        new_expression = ch + operand1 + operand2

        stack.append(new_expression)

    return stack[-1]

postfix = "ABC*+"

print("Postfix:", postfix)
print("Prefix:", postfix_to_prefix(postfix))

```

Output:

```

Postfix: ABC*+
Prefix: +A*BC

```

10. Prefix → Postfix

Prefix:

```
*+ABC
```

Scan from **right to left**.

For an operator:

```

operand1 = POP
operand2 = POP

result = operand1 + operand2 + operator

PUSH result

```

Python Code

Python

Run

```

def postfix_to_prefix(expression):
    stack = []

    for ch in expression[::-1]:

```

```

        if ch.isalnum():
            stack.append(ch)

        else:
            operand1 = stack.pop()
            operand2 = stack.pop()

            new_expression = operand1 + operand2 + ch

            stack.append(new_expression)

    return stack[-1]

prefix = "*+ABC"

print("Prefix:", prefix)
print("Postfix:", prefix_to_postfix(prefix))

```

Output:

```

Prefix: *+ABC
Postfix: AB+C*

```

11. All Expression Conversions

The following table is useful for exams.

Conversion	Direction	Stack Traversal
Infix $\rightarrow$ Postfix	Left $\rightarrow$ Right	L $\rightarrow$ R
Infix $\rightarrow$ Prefix	Reverse + Postfix	—
Postfix $\rightarrow$ Infix	Left $\rightarrow$ Right	L $\rightarrow$ R
Prefix $\rightarrow$ Infix	Right $\rightarrow$ Left	R $\rightarrow$ L
Postfix $\rightarrow$ Prefix	Left $\rightarrow$ Right	L $\rightarrow$ R
Prefix $\rightarrow$ Postfix	Right $\rightarrow$ Left	R $\rightarrow$ L

12. Complexity Analysis of Expression Conversion

Let n be the number of symbols.

Operation	Time	Auxiliary Space
Infix $\rightarrow$ Postfix	$O(n)$	$O(n)$
Infix $\rightarrow$ Prefix	$O(n)$	$O(n)$
Postfix $\rightarrow$ Infix	$O(n)$	$O(n)$
Prefix $\rightarrow$ Infix	$O(n)$	$O(n)$
Postfix $\rightarrow$ Prefix	$O(n)$	$O(n)$
Prefix $\rightarrow$ Postfix	$O(n)$	$O(n)$

Why $O(n)$?

Every symbol is:

- scanned a constant number of times,
- pushed at most once,
- popped at most once.

Therefore:

Time Complexity = $O(n)$
 Space Complexity = $O(n)$

13. Combined Example

Consider:

$A + B * (C - D)$

Infix

$A+B*(C-D)$

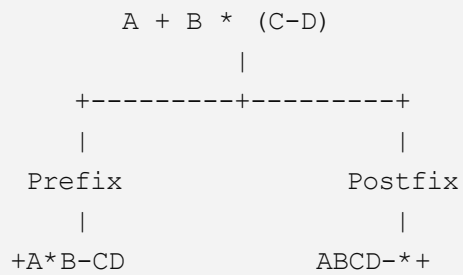
Postfix

$ABCD-*+$

Prefix

+A*B-CD

So:



14. Important Exam Comparison

NGE

Traverse: Right $\rightarrow$ Left

```
while stack.top <= current:
    pop
```

```
answer = stack.top
push current
```

PSE

Traverse: Left $\rightarrow$ Right

```
while stack.top >= current:
    pop
```

```
answer = stack.top
push current
```

Infix $\rightarrow$ Postfix

Operand $\rightarrow$ Output

'(' $\rightarrow$ Push

')' $\rightarrow$ Pop until '('

Operator $\rightarrow$

Pop higher/equal precedence operators

Push current operator

Postfix Evaluation

Operand → Push

Operator →

b = Pop()

a = Pop()

result = a operator b

Push(result)

15. One-Line Memory Tricks

NGE → Right side, Greater → scan Right-to-Left

PSE → Left side, Smaller → scan Left-to-Right

Infix → Operator is BETWEEN operands

Prefix → Operator is BEFORE operands

Postfix → Operator is AFTER operands

The most important stack pattern to master is the **monotonic stack** used in NGE/PSE: instead of comparing an element with every other element, we discard elements that can never become an answer. This is what reduces the running time from **$O(n^2)$ to $O(n)$** .